

Atividade Prática: Refatoração de Múltiplos Componentes Legados (SRP e OCP)

1. Contextualização do Cenário Corporativo

Nesta atividade vocês atuarão no papel de Engenheiros de Software encarregados de mitigar a dívida técnica de um Sistema de Recursos Humanos e Folha de Pagamento. O sistema, outrora monolítico e incipiente, sofreu grave degradação estrutural devido à constante adição de requisitos sem o devido planejamento arquitetural.

A diretoria de Recursos Humanos emitiu um memorando com as seguintes exigências para o próximo ciclo de desenvolvimento:

1. Implementação de novos níveis hierárquicos (Ex: "Especialista", "Consultor Externo"), cada qual com regras próprias de elegibilidade a benefícios.
2. Necessidade de exportar relatórios gerenciais não apenas em tela e PDF, mas também em formato XML para integração governamental.
3. Substituição iminente do banco de dados relacional atual por uma infraestrutura baseada em nuvem.

A análise preliminar do código-fonte revelou um alto grau de acoplamento e múltiplas violações dos Princípios da Responsabilidade Única (SRP) e do Aberto/Fechado (OCP).

2. O Desafio Estrutural (Artefatos Legados)

Abaixo, encontram-se três classes centrais do sistema. Elas operam de forma interdependente e apresentam falhas severas de design. A missão dos alunos é diagnosticar e refatorar este ecossistema.

Artefato 1: CalculadoraBeneficios.java

```
public class CalculadoraBeneficios {  
  
    public double calcularValeAlimentacao(String cargo, double salarioBase) {  
        if (cargo.equalsIgnoreCase("ESTAGIARIO")) {  
            return 250.00;  
        } else if (cargo.equalsIgnoreCase("JUNIOR") || cargo.equalsIgnoreCase("PLENO")) {  
            return 600.00;  
        } else if (cargo.equalsIgnoreCase("SENIOR") || cargo.equalsIgnoreCase("DIRETOR")) {
```

```

        return 850.00;
    }
    throw new IllegalArgumentException("Cargo não reconhecido na matriz de benefícios.");
}

public double calcularAuxilioSaude(String cargo) {
    if (cargo.equalsIgnoreCase("DIRETOR")) {
        return 1500.00;
    }
    return 300.00;
}
}

```

Artefato 2: GeradorRelatorioRH.java

```

import java.io.FileWriter;
import java.io.IOException;

public class GeradorRelatorioRH {

    public void emitirRelatorioMensal(String nomeFuncionario, double salarioLiquido, String formato) {
        if (formato.equalsIgnoreCase("PDF")) {
            System.out.println("Iniciando conversão binária para PDF...");
            System.out.println("Documento PDF gerado: Funcionario: " + nomeFuncionario + " | Salario: " +
salarioLiquido);
        } else if (formato.equalsIgnoreCase("CSV")) {
            try {
                FileWriter csvWriter = new FileWriter("relatorio_rh.csv", true);

                csvWriter.append(nomeFuncionario).append(";").append(String.valueOf(salarioLiquido)).append(
d("\n"));
                csvWriter.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        } else {
            System.out.println("Formato não suportado para emissão.");
        }
    }
}

```

Artefato 3: ProcessadorFolhaPagamento.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class ProcessadorFolhaPagamento {

    public void processarPagamento(String cpf, String nome, String cargo, double salarioBase, String
formatoRelatorio) {

        CalculadoraBeneficios calcBeneficios = new CalculadoraBeneficios();
        double va = calcBeneficios.calcularValeAlimentacao(cargo, salarioBase);
        double saude = calcBeneficios.calcularAuxilioSaude(cargo);

        double descontos = salarioBase * 0.11;
        double salarioLiquido = (salarioBase + va + saude) - descontos;

        try {
            Connection conn = DriverManager.getConnection("jdbc:oracle:thin:@servidor_bd:1521:rh",
"sysdba", "senha_forte");
            String sql = "INSERT INTO pagamentos (cpf, valor_liquido, status) VALUES (?, ?, 'PROCESSADO)";
            PreparedStatement stmt = conn.prepareStatement(sql);
            stmt.setString(1, cpf);
            stmt.setDouble(2, salarioLiquido);
            stmt.executeUpdate();
            conn.close();
        } catch (Exception e) {
            e.printStackTrace();
            return;
        }

        GeradorRelatorioRH relatorio = new GeradorRelatorioRH();
        relatorio.emitirRelatorioMensal(nome, salarioLiquido, formatoRelatorio);

        System.out.println("Enviando requisição de transferência via API REST para o Banco Central...");
        System.out.println("{ \"cpf\": \"" + cpf + "\", \"valor\": " + salarioLiquido + " }");
    }
}
```

3. Roteiro de Execução

Vocês deverão produzir um documento técnico e a respectiva refatoração em código-fonte, englobando as seguintes etapas:

Fase 1: Diagnóstico Arquitetural

1. Identifiquem onde a classe `CalculadoraBeneficios` fere o Princípio do Aberto/Fechado (OCP). Demonstrem teoricamente o impacto de adicionar os cargos "Especialista" e "Consultor Externo".
2. Analisem a classe `GeradorRelatorioRH`. Quais princípios estão sendo violados simultaneamente?
3. Na classe `ProcessadorFolhaPagamento`, apontem as violações do Princípio da Responsabilidade Única (SRP) e expliquem por que a inicialização direta de instâncias (`new CalculadoraBeneficios()`, `new GeradorRelatorioRH()`) gera um Acoplamento Patológico.

Fase 2: Refatoração Estrutural (Modelagem e Código)

- **Abstração de Benefícios (OCP):** Utilizem o padrão *Strategy* para criar contratos (Interfaces) referentes às políticas de benefícios. Cada política deve ser uma classe distinta, eliminando o uso de estruturas condicionais `if/else` baseadas em texto.
- **Abstração de Formatação (OCP e SRP):** Criem uma interface para a geração de relatórios, isolando a lógica de formatação de strings e de manipulação de I/O (Arquivos).
- **Segregação de Infraestrutura (SRP):** Extraiam a lógica de persistência de dados (JDBC) e a lógica de comunicação externa (API do Banco) para repositórios e serviços de integração específicos.
- **Inversão de Dependência:** Refatorem a classe orquestradora (`ProcessadorFolhaPagamento`) para que não instancie suas próprias dependências, mas sim as receba via construtor (Injeção de Dependência).